



**THE UNITED REPUBLIC OF TANZANIA
e-GOVERNMENT AUTHORITY**



DAIRE SYSTEM REQUIREMENT SPECIFICATION

Version 1.0

**e-Government Research, Innovation, and Development Center
e-GovRIDC**

DAIRE- Software Requirements Specification

Version: 1.0

Prepared by: DAIRE Team

1. Amina Kalonge (IFM)
2. Atanasi Kafuka (UDOM)
3. Amiri Athumani (MZUMBE)
4. Anthony Athanas (UDOM)
5. Eves Vitus (UDOM)
6. John Nabwanda (UDOM)

Supervisors: Jackson Bakari, Francis Elias, Noel Jackson, Mkadam Msabaha

Organization: e-Government Authority- RIDC

Project: DAIRE – A Blockchain-Based AI Reputation Engine for Decentralized Credit Assessment

Client: Financial Service Providers (CRDB, NMB and MoFinance) and their customers

Date Created: October 08, 2026

Project Reference: eGA/DAIRE/2026/001

Document Control:

1. Document ID: eGA/DAIRE/SRS/001/2026
2. Security Classification: Official
3. Distribution: eGA-RIDC, Lender Institutions, DAIRE Team

Revision History

Name	Date	Reason For Changes	Version
DAIRE Team	08/10/2026	Initial SRS Creation	1.0

Table of Contents

1. Introduction.....	1
1.1 Purpose	1
1.2 Document Conventions.....	1
1.3 Intended Audience.....	2
1.3.1 Primary Audience	2
1.3.2 Secondary Audience	2
1.3.3 Reading Recommendations	2
1.4 Project Scope.....	3
1.4.1 In-Scope Features.....	3
1.4.2 Out-of-Scope Features.....	3
1.4.3 Key Benefits.....	4
1.4.4 Strategic Alignment.....	4
1.5 References.....	4
1.5.1 eGA Standards and Guidelines.....	4
1.5.2 Legal and Regulatory Framework.....	4
1.5.3 Technical Standards.....	4
1.5.4 Project Documentation.....	5
1.5.5 Industry Standards.....	5
2. Overall Description.....	5
2.1 Product Perspective.....	5
2.1.1 System Architecture Context.....	6
2.1.2 Market Position & Value Proposition.....	7
2.1.3 Ecosystem Relationships & Compatibility.....	7
2.2 Product Functions.....	7
2.2.1 Customer Onboarding and Identity Verification	7
2.2.2 Core Banking Operations.....	7
2.2.3 Loan Lifecycle Management.....	7
2.2.4 Consent and Data Sharing.....	7
2.2.5 Central Data Exchange and Orchestration.....	7
2.2.6 Credit Scoring and Result Broadcast.....	8
2.3 User Classes, Characteristics, and Needs.....	8
2.3.1 User Access Levels.....	8
2.3.2 Primary Users: Customers (Borrowers).....	8
2.3.3 Secondary Users: Lender Administrators and Loan Officers.....	8
2.3.4 Supporting Users.....	9
2.4 Operating Environment.....	9
2.4.1 Client Platform Requirements.....	9
2.4.2 Network and Connectivity	9
2.4.3 Integration Environment.....	9
2.4.4 Server and Deployment Environment.....	9
2.4.5 Security and Compliance.....	9
2.5 Design and Implementation Constraints.....	10
2.5.1 Technical Constraints.....	10
2.5.2 Regulatory Constraints.....	10
2.5.3 Business Constraints	10
2.6 User Documentation.....	10
2.6.1 Customer and Staff Documentation.....	10
2.6.2 Technical Documentation.....	11

2.7 Assumptions and Dependencies.....	11
2.7.1 Technical Assumptions.....	11
2.7.2 Business Assumptions.....	12
2.7.3 External Dependencies.....	12
3. System Features.....	12
3.1 Module 1: Authentication & Identity Verification.....	12
3.1.1 Customer Registration.....	12
3.1.2 NIDA Pre-Verification.....	13
3.1.3 User Login.....	14
3.1.4 PIN Setup and Password Reset.....	15
3.1.5 Profile Management.....	15
3.2 Module 2: Accounts, Transactions & Savings.....	16
3.2.1 Account Management.....	16
3.2.2 Cash Deposit.....	17
3.2.3 Cash Withdrawal.....	17
3.2.4 Fund Transfer.....	18
3.2.5 Savings Management.....	19
3.2.6 Group Savings Management.....	19
3.3 Module 3: Loan Management.....	20
3.3.1 Loan Application.....	20
3.3.2 Loan Review and Approval.....	20
3.3.3 Loan Rejection.....	21
3.3.4 Repayment Schedule and Payment.....	22
3.3.5 Credit History and Inquiries.....	22
3.4 Module 4: Consent & Data Sharing.....	23
3.4.1 Grant Consent.....	23
3.4.2 Revoke and Extend Consent.....	24
3.4.3 Consent-Based Data Category Enforcement.....	24
3.5 Module 5: Lender–Central Integration.....	25
3.5.1 Push Customer Data to Central.....	25
3.5.2 Receive Central Assessment Result.....	25
3.5.3 Central Borrower Lookup.....	26
3.5.4 Synchronisation Logs and Retry.....	27
3.5.5 View Central Credit Score.....	27
3.6 Module 6: Central System Operations.....	28
3.6.1 Lender Registration and Status.....	28
3.6.2 Receive and Merge Lender Data.....	28
3.6.3 Pull Borrower Data from All Lenders.....	29
3.6.4 Data Routing Policy.....	30
3.6.5 Data Exchange Audit.....	30
3.6.6 Operations Dashboard and Network Status.....	31
3.7 Module 7: AI Reputation & Blockchain Scoring.....	31
3.7.1 Create Assessment.....	31
3.7.2 AI Reputation Analysis.....	32
3.7.3 Blockchain Credit Score Calculation.....	33
3.7.4 Broadcast Assessment Result.....	33
3.8 Module 8: NIDA Registry, Reporting & Administration.....	34
3.8.1 NIDA Identity Verification Service.....	34
3.8.2 NIDA Lookup, Health and Registry Listing.....	35
3.8.3 Reports and Analytics.....	35

3.8.4 Audit Logs and Notifications.....	36
3.8.5 System Settings and Lender Credentials.....	36
4. External Interface Requirements.....	37
4.1 User Interfaces.....	37
4.1.1 Authentication Screens (all lender systems).....	37
4.1.2 Customer Dashboard.....	37
4.1.3 Lender Administrator Dashboard.....	37
4.1.4 Central System Operations Console.....	38
4.1.5 Common Interface Requirements.....	38
4.2 Hardware Interfaces.....	38
4.3 Software Interfaces.....	38
4.4 Communication Interfaces.....	39
5. Non-Functional Requirements.....	39
5.1 Performance Requirements.....	39
5.1.1 Response Time Requirements.....	39
5.1.2 Throughput Requirements.....	39
5.1.3 Resource Utilization.....	39
5.1.4 Scalability Requirements.....	40
5.2 Security Requirements.....	40
5.2.1 Authentication and Authorization.....	40
5.2.2 Data Protection.....	40
5.2.3 Privacy and Compliance.....	41
5.2.4 Security Monitoring and Incident Response.....	41
5.3 Reliability and Availability.....	41
5.3.1 Availability Requirements.....	41
5.3.2 Fault Tolerance.....	41
5.3.3 Disaster Recovery.....	42
5.3.4 Error Handling and Recovery.....	42
5.4 Usability and Accessibility.....	42
5.4.1 User Interface Usability.....	42
5.4.2 Accessibility Standards.....	42
5.4.3 Multilingual Support.....	42
5.4.4 User Experience Optimization.....	43
5.5 Maintainability and Portability.....	43
5.5.1 Maintainability Requirements.....	43
5.5.2 Portability Requirements.....	43
5.5.3 Compatibility Requirements.....	43
5.6 Legal and Compliance Requirements.....	44
5.6.1 Regulatory Compliance.....	44
5.6.2 Intellectual Property and Licensing.....	44
5.6.3 Service Level Agreements.....	44
5.7 Operational Requirements.....	44
5.7.1 Monitoring and Logging.....	44
5.7.2 Backup and Recovery Operations.....	44
5.7.3 System Administration.....	44
5.7.4 Documentation Requirements.....	45
5.8 Cultural and Political Requirements.....	45
5.8.1 Cultural Considerations.....	45
5.8.2 Political and Governance Requirements.....	45
6. Other Requirements.....	45

6.1 Data Migration Requirements.....	45
6.2 Internationalization Requirements.....	46
6.3 Training Requirements.....	46
6.4 Performance Requirements.....	46
6.5 Safety Requirements.....	46
6.6 Security Requirements.....	47
6.7 Software Quality Attributes.....	47
6.8 Business Rules.....	47
7. Appendix.....	48
1. Glossary.....	48

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document specifies the comprehensive requirements for the Decentralized AI Reputation Engine (DAIRE), Version 1.0, a blockchain-based AI reputation engine for decentralized credit assessment, developed under the e-Government Authority Research, Innovation and Development Center (e-GovRIDC) of the United Republic of Tanzania.

DAIRE calculates and provides credit scores for individuals and groups such as entrepreneurs, small groups of women and youth, and small-scale farmers, who often lack a formal credit history. The platform brings together the lender systems of Financial Service Providers (FSPs), the national identity registry (NIDA), a Central System that unifies customer data, an AI reputation model and a blockchain smart contract that computes the credit score.

This SRS covers the complete system as implemented: the four lender systems (CRDB-01, MoFinance-004, NMB-01 and NMB-02), the NIDA registry system, the DAIRE Central System (backend and operations console), the middleware and smart contract used for blockchain scoring, and the AI engine used for reputation analysis.

1.2 Document Conventions

This Software Requirements Specification (SRS) adheres to a defined set of conventions and standards to ensure consistency, clarity, and ease of understanding across all sections. These conventions serve as a uniform guideline for representing requirements, formatting, and references throughout the document.

1. Requirement Identifiers

- I. **Functional Requirements (FR):** All functional requirements follow the format FR-[Category]-[Number] (e.g., FR-LOAN-001).
- II. **Non-Functional Requirements (NFR):** Non-functional requirements use the format NFR-[Category]-[Number] (e.g., NFR-PERF-001).
- III. **Use Cases (UC):** Each use case is labeled as UC-[Number] (e.g., UC-001).

2. Priority Levels

Each requirement is assigned a priority level based on its criticality to system functionality:

- I. High (H): Essential for core functionality and system success.
- II. Medium (M): Important but not critical; may be deferred if necessary.
- III. Low (L): Enhances usability or efficiency but not essential.

3. Compliance Standards

All requirements must comply with the following:

- I. e-Government Authority (eGA) standards and e-Government Enterprise Architecture (e-GEA) framework.
- II. Tanzania Data Protection Act, 2022.
- III. Electronic Transactions Act, 2015 and Electronic and Postal Communications Act (EPOCA), 2010.

4. Language and Terminology

- I. Technical terms, abbreviations, and acronyms are standardized and defined in Appendix A: Glossary.
- II. Consistent use of domain-specific terminology is maintained to ensure no ambiguity (for example, "borrower reference", "assessment reference", "lender", "Central System").

5. Typography and Font Styles

- I. **Bold Text:** Used for section headings and critical requirements.

- II. *Italic Text*: Used for references to external documents, standards, or supplementary notes.
- III. *Times New Roman*: Used for technical terms, variable names, and user inputs.

6. Numbering and Sectioning

- I. A hierarchical numbering system is used (e.g., 1. Introduction, 1.1 Purpose, 1.2 Scope).
- II. Subsections use decimal notation (e.g., 3.1, 3.2, 3.2.1) to indicate their relationship to parent sections.

7. Tables, Diagrams, and Charts

- I. Tables are used to present structured information such as requirements lists, data mappings, and priority matrices.
- II. Diagrams (e.g., the DAIRE conceptual diagram) are applied to represent workflows, interactions, and system structures.
- III. All visual elements follow a consistent format and labeling convention for clarity.

1.3 Intended Audience

This document is intended for a diverse group of stakeholders involved in the design, development, implementation, and usage of the DAIRE platform.

1.3.1 Primary Audience

The primary stakeholders are directly responsible for the development, operation, and maintenance of the system:

- 1. Development Team and Supervisors
 - a. Software developers, blockchain and AI engineers, and quality assurance staff who build and test the lender systems, NIDA system, Central System, smart contract and AI model.
- 2. eGA Technical Staff
 - a. Project managers, technical reviewers, and compliance officers ensuring adherence to e-Government Authority (eGA) standards and frameworks.
- 3. Financial Service Provider (FSP) Management and IT Personnel
 - a. Bank and microfinance operations staff and IT administrators responsible for connecting a lender system to DAIRE and for daily operation.

1.3.2 Secondary Audience

The secondary stakeholders support the project indirectly by ensuring compliance, governance, and usability:

- 1. Regulators and Compliance Teams
 - a. Teams ensuring adherence to national regulations, including the Tanzania Data Protection Act, 2022, and sector-specific rules.
- 2. National Identification Authority (NIDA) Representatives
 - a. Stakeholders interested in how the NIDA number is verified and used as the cross-lender identity key.
- 3. End Users (Customers and Loan Officers)
 - a. Bank customers and loan officers who may review documentation during user acceptance testing.

1.3.3 Reading Recommendations

Different stakeholders are encouraged to focus on specific sections of the document relevant to their roles:

- 1. Development Teams: Sections 3–5 (Functional and Non-Functional Requirements, Technical Specifications).

2. Project Managers and Supervisors: Sections 1–2 and 6 (Introduction, Scope, System Constraints, and Management Overview).
3. Compliance Officers and Regulators: Sections 5.2, 5.6 and 6 (Security, Data Protection, and Regulatory Compliance).
4. Lender Integrators: Sections 3.5, 4.3 and 4.4 (Lender–Central integration, software and communication interfaces).

1.4 Project Scope

The DAIRE project encompasses the design, development, and deployment of a reputation engine that calculates and provides credit scores using blockchain technology and AI for evaluation. It improves access to credit for small-scale farmers, youth, women, and entrepreneurs who struggle to obtain suitable loans because of the lack of a credit score system and of accurate assessment of how responsibly they can be credited.

The system integrates the lender systems with a Central System (Web2), connects the Central System to a blockchain (Web3) that calculates the credit score, and uses an AI model to provide reputation and risk recommendations. The project involves the following boundaries and key features:

1.4.1 In-Scope Features

1. Lender Systems
 - a. Four lender systems built from one lender template: CRDB-01, MoFinance-004, NMB-01 and NMB-02, each with its own user interface.
 - b. Customer registration, accounts, transactions, savings, loans, repayments, consent and reporting.
2. NIDA System
 - a. A mock national registry that verifies the 20-digit NIDA number and names and returns the citizen record.
3. Central System (Web2)
 - a. Lender registry, data ingestion and merging, borrower identity matching, routing policy, data-exchange audit and operations console.
4. Web2–Web3 Integration
 - a. Middleware that submits derived features to the smart contract and reads the score back.
5. Blockchain Credit Score
 - a. DaireCreditScore smart contract computing a score from five dimensions (D1–D5).
6. AI Reputation Model
 - a. Random Forest model that returns default probability, reputation and risk level.

1.4.2 Out-of-Scope Features

1. Real integration with the live NIDA production system (a mock registry with synthetic citizens is used).
2. Real mobile-money, card or interbank payment gateways; transactions are processed inside each lender system.
3. A native mobile application; all interfaces are web-based (Angular).
4. Production-grade deployment on a public blockchain; the smart contract is deployed on a local Hardhat network.
5. Automatic lending decisions; each lender applies its own policy and staff approve loans.

1.4.3 Key Benefits

1. Financial inclusion: citizens without a bank credit history can still obtain a reputation score.

2. Responsible lending: citizens borrow an amount they can handle and lenders give loans accordingly.
3. Tamper-proof scoring: the score is computed by a deterministic smart contract that anyone can audit.
4. Cross-lender view: one score built from the data of all lenders that serve the same borrower.
5. Privacy by design: no personal data is written on-chain, only derived integers.

1.4.4 Strategic Alignment

This project directly supports:

1. eGA's mission of fostering ICT adoption across public institutions.
2. Tanzania's digital transformation and financial inclusion goals for small-scale farmers, women and youth.
3. The use of national identity (NIDA) as a trusted foundation for digital services.
4. Research, innovation and digital development promoted by e-GovRIDC.

1.5 References

This section lists the documents, standards, and regulatory frameworks referenced in the preparation of the DAIRE SRS. These references provide technical, regulatory, and industry guidelines that govern the system's design, implementation, and compliance requirements.

1.5.1 eGA Standards and Guidelines

Ref. No.	Title	Document Code / Link
[EGA-001]	e-Government Architecture Vision – Standards and Technical Guidelines	eGA/EXT/AVS/001
[EGA-002]	Guidebook for Formulating and Governing Enterprise Architecture in Public Institutions	eGA/EXT/AVS/004
[EGA-003]	Government Hardware and Software Specifications	eGA/EXT/IRA/005

1.5.2 Legal and Regulatory Framework

Ref. No.	Title	Year	Citation Code
[LAW-001]	Tanzania Data Protection Act	2022	TDP Act 2022
[LAW-002]	Electronic and Postal Communications Act	2010	EPOCA 2010
[LAW-003]	Electronic Transactions Act	2015	ETA 2015

1.5.3 Technical Standards

Ref. No.	Title	Standard Code
[STD-001]	Information Security Management	ISO/IEC 27001:2013
[STD-002]	Systems and Software Quality Models (SQuaRE)	ISO/IEC 25010:2011
[STD-003]	JSON Web Token (JWT)	RFC 7519
[STD-004]	OpenAPI Specification (API schema at /api/schema/)	OpenAPI 3.0

1.5.4 Project Documentation

Ref. No.	Title	Document Code / Link
[DOC-001]	DAIRE Conceptual Diagram	Project presentations (Sept–Oct

Ref. No.	Title	Document Code / Link
		2026)
[DOC-002]	DAIRE Central System README and API reference	central_system/DAIRE/README.md
[DOC-003]	AI Data Contract (JSON in / JSON out)	central_system/DAIRE/AI_DATA_CONTRACT.md
[DOC-004]	DAIRE Lender Subsystem Template	central_system/DAIRE/LENDER_SUBSYSTEM_README.md
[DOC-005]	NMB ↔ Central System Integration Report	nmb/INTEGRATION_REPORT.md
[DOC-006]	Architecture – Where the Credit Score Is Calculated	central_system/docs/ARCHITECTURE.md
[DOC-007]	Credit score contract and formula (STEP-02)	central_system/docs/STEP-02-SCORING.md

1.5.5 Industry Standards

Ref. No.	Title	Standard Code
[IND-001]	Recommended Practice for Software Requirements Specifications	IEEE 830-1998
[IND-002]	FICO risk bands (POOR, FAIR, GOOD, VERY_GOOD, EXCEPTIONAL) used for the score range 300–850	FICO score bands
[IND-003]	Min-max normalization for credit scoring	arXiv:2412.00710v2

2. Overall Description

2.1 Product Perspective

DAIRE is a system of cooperating systems rather than a single application. Financial Service Providers (FSPs) keep their private customer, account, transaction, loan and repayment data in their own lender systems. The Central System receives the data that each customer has consented to share, merges it by the borrower's identity, and sends only derived, non-identifying values to the AI engine and to the blockchain. The result is one credit score and one reputation analysis, which the Central System returns to every lender that serves the borrower.

The National Identification Authority (NIDA) system is consulted by each lender at registration time: the lender sends a data request (NIDA number and names) and receives verified data. The NIDA number then becomes the key that identifies the same person across lenders.

DAIRE CONCEPTUAL DIAGRAM

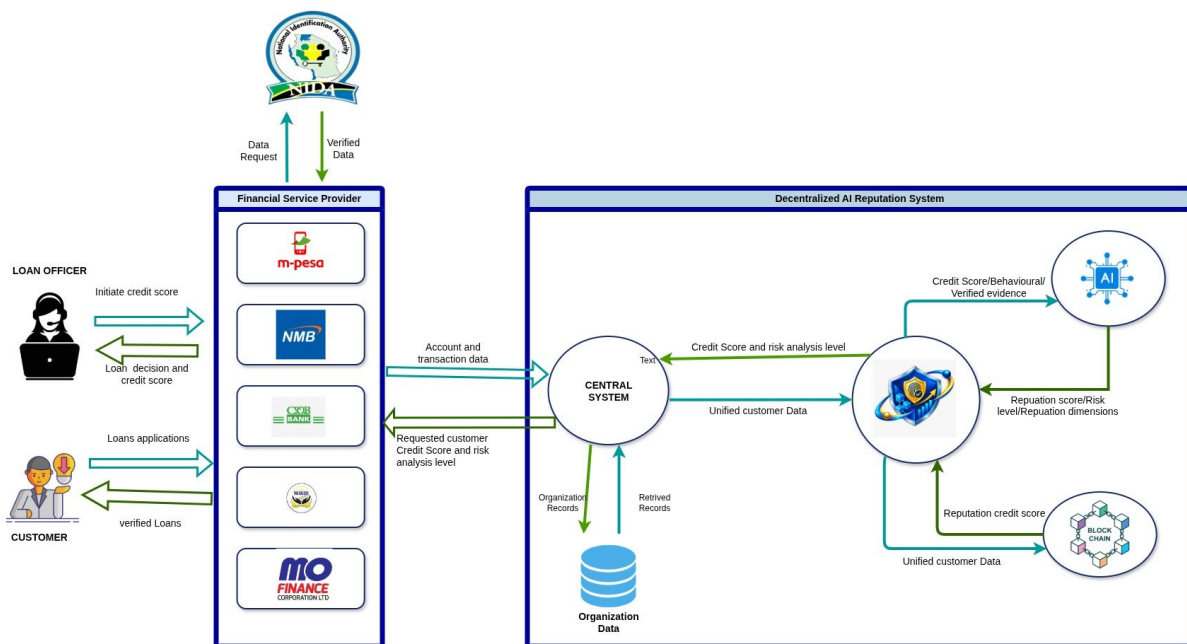


Figure 2.1: DAIRE conceptual diagram

2.1.1 System Architecture Context

1. Architecture Model
 - a. Federated system of systems: lender systems, NIDA system, Central System, middleware, blockchain and AI engine.
 - b. Each lender is a duplicate of one lender template with its own database and user interface.
2. Technology Stack
 - a. Frontend: Angular (customer, lender admin and Central operations consoles).
 - b. Backend: Django and Django REST Framework (Python); middleware in Node.js (Express).
 - c. Storage: PostgreSQL for every lender, for NIDA and for the Central System.
 - d. Blockchain: Solidity smart contract (DaireCreditScore) on a Hardhat local network.
 - e. AI: scikit-learn Random Forest pipeline stored as a joblib file.
3. Integration Layer
 - a. REST APIs with JSON payloads between all systems.
 - b. Lender → Central push and Central → lender broadcast.
 - c. Lender → NIDA verification and lookup.
4. Security Layer
 - a. JWT authentication (12-hour access token, 7-day refresh token) for users.
 - b. Hashed 4-digit PIN with one-time password at customer login.
 - c. Trusted source IP whitelists between systems (TRUSTED_CLIENT_IPS for NIDA, TRUSTED_CALLBACK_IPS for lenders).
 - d. Customer consent enforced before any data leaves a lender.
5. Data Protection Controls
 - a. Only data categories consented to by the customer are sent to Central.
 - b. Bank-only fields (NIDA number, phone, email, address, next of kin) are excluded from the outbound payload.

- c. A routing policy limits which fields reach the AI and the blockchain.

2.1.2 Market Position & Value Proposition

1. Unique Value Proposition
 - a. A reputation score for people with no formal credit history.
 - b. One score shared by all lenders that serve the same borrower.
2. Competitive Positioning
 - a. Unlike traditional credit bureaus, the score is computed by a smart contract whose rules anyone can verify.
 - b. Weights are adapted to Tanzania: mobile-money style activity and savings weigh more than formal bank loans.
3. Target Users
 - a. Small-scale farmers, youth, women groups and entrepreneurs.
 - b. Banks, microfinance institutions and mobile-money lenders.

2.1.3 Ecosystem Relationships & Compatibility

1. NIDA System – verifies identity; the NIDA number is the global customer key.
2. Lender systems (CRDB-01, MoFinance-004, NMB-01, NMB-02) – push data and receive results.
3. Central System – merges lender data and orchestrates scoring and broadcast.
4. AI engine and blockchain – return the reputation analysis and the credit score.
5. Potential future extensions: further lenders such as mobile-money operators shown in the conceptual diagram (e.g., m-pesa), using the same lender template.

2.2 Product Functions

The DAIRE platform provides the following core functional categories:

2.2.1 Customer Onboarding and Identity Verification

1. Verify the NIDA number and names against the NIDA registry before a customer is registered.
2. Create the user, customer profile, primary savings account and pending consent in one step.
3. Support login with account number, PIN and a one-time password.

2.2.2 Core Banking Operations

1. Manage accounts, deposits, withdrawals and transfers.
2. Manage savings records and group savings (members and contributions).
3. Record the financial behaviour and credit history of each customer.

2.2.3 Loan Lifecycle Management

1. Accept loan applications, review them, approve with a rate and amount, disburse and reject or cancel.
2. Generate the repayment schedule and record repayments.

2.2.4 Consent and Data Sharing

1. Let the customer grant, revoke and extend consent per data category.
2. Block any transfer to Central without an active consent.

2.2.5 Central Data Exchange and Orchestration

1. Register lenders and track their connection status (CONNECTED, DEGRADED, DISCONNECTED).
2. Receive pushed lender data or pull a borrower from all lenders concurrently.

3. Merge data by borrower reference, keep each source lender, and record conflicts.
4. Audit every inter-system exchange in the data exchange log.

2.2.6 Credit Scoring and Result Broadcast

1. Build 11 normalized credit features and five dimensions (D1–D5) from the merged data.
2. Run the AI model to obtain the default probability, reputation and risk level.
3. Run the blockchain scoring to obtain the credit score and transaction hash.
4. Broadcast the same assessment-specific result to every lender linked to the borrower.
5. Display the result to the loan officer and to the customer.

Reporting, audit logging, notifications and system settings support all of the above in each lender system and in the Central System.

2.3 User Classes, Characteristics, and Needs

DAIRE serves several user classes with different privileges and technical skills.

2.3.1 User Access Levels

Role	Access Level	Interface	Key Permissions
Customer	Standard (role = customer)	Lender customer dashboard	Own profile, accounts, transactions, savings, loan applications, repayments, consent and received credit score.
Lender Administrator / Loan Officer	Administrative (role = admin)	Lender admin dashboard	All customers, loan approval and rejection, repayments, reports, audit logs, push to Central, settings.
Central Operator	Session login (Django)	Central operations console	Lenders, borrowers, data exchange, assessments, routing policy and broadcast.
Integrating System	Trusted IP	REST API	NIDA verification (lenders), receive result (lenders), receive data (Central).
Guest	No account	Registration and login pages	Registration, NIDA pre-verification, login only.

2.3.2 Primary Users: Customers (Borrowers)

1. Characteristics: individuals and group members such as farmers, youth, women entrepreneurs; basic digital literacy; use a web browser on a phone or computer.
2. Needs: register with the NIDA number, save, transfer and borrow, understand their credit score, and control who may see their data.
3. Expectations: simple screens, quick responses and clear error messages.

2.3.3 Secondary Users: Lender Administrators and Loan Officers

1. Characteristics: bank or microfinance staff with training in the lender admin dashboard.
2. Needs: review loan applications with the Central credit score and risk analysis, approve or reject, disburse, monitor repayments and defaults, and push customer data to Central with consent.
3. Expectations: accurate reports and an audit trail of every decision.

2.3.4 Supporting Users

1. Central System operators who run assessments and broadcast results.
2. NIDA registry administrators who maintain citizen records through the Django admin.
3. System integrators and DevOps staff who deploy and connect lenders.
4. Auditors and supervisors who review the audit logs and the data exchange history.

2.4 Operating Environment

2.4.1 Client Platform Requirements

1. Modern web browsers (Chrome, Firefox, Edge, Safari) able to run an Angular single-page application.
2. Network access to the lender or Central web address on the local network.
3. No installation is required on the client device.

2.4.2 Network and Connectivity

All systems run on a trusted local network (172.17.16.x in the demonstration environment) and communicate over HTTP using JSON.

System	Address / Port	Role
Central System console	172.17.16.68:4200	Angular operations console
Central backend	port 8000	Django REST API
Middleware	port 5000	Express gateway to the smart contract
Blockchain node	port 8545	Hardhat local network
CRDB backend / UI	172.17.16.70:8002 / :4600	Lender CRDB-01
NMB backend / UI	172.17.16.76 : 8002 / :4300	Lenders NMB-01 and NMB-02
NIDA System	172.17.16.70:8020	Identity registry API

2.4.3 Integration Environment

1. Lender → NIDA: POST /api/v1/nida/verify/ at registration.
2. Lender → Central: POST /api/lender-data/receive/ with the consented customer document.
3. Central → Lender: POST /api/daire/central/receive/ with the credit result.
4. Central → AI and Central → blockchain middleware for scoring.

2.4.4 Server and Deployment Environment

1. Linux hosts running Python 3 virtual environments for each Django project.
2. PostgreSQL databases: nida_registry, the lender databases, and the Central database (Docker container on port 5434).
3. Django servers bound to 0.0.0.0 so other machines on the network can reach them.
4. Hardhat node and Express middleware started with the Central start-all script.
5. Angular development servers for the user interfaces (ports 4200, 4300 and 4600).

2.4.5 Security and Compliance

1. Systems authenticate each other by trusted source IP; staff authenticate with JWT or Django sessions.
2. Customer consent is checked before data leaves a lender.
3. Personal data stays in PostgreSQL; only derived integers go to the blockchain.

4. The network is private; HTTP is used inside the trusted network and HTTPS/VPN is required beyond it.

2.5 Design and Implementation Constraints

2.5.1 Technical Constraints

1. Technology constraints
 - a. The lender, NIDA and Central backends use Python and Django REST Framework (Django 4.2.16 for NIDA and the lenders).
 - b. The user interfaces are built with Angular; the middleware with Node.js.
2. Identity constraint
 - a. The NIDA number is 20 digits (YYYYMMDD plus 12 digits) and is the global key.
3. Network constraint
 - a. All systems must be reachable on the same private network; the NIDA whitelist and the lender callback whitelist must contain each caller's IP address.
1. Blockchain constraint
 - a. The contract is deployed on a local Hardhat network; gas cost and public availability are not evaluated.
 - b. Only derived integers (basis points and counts) can be sent to the contract.
2. AI constraint
 - a. The local model uses 11 features; some training columns (home ownership, employment length, loan intent, loan grade) are not collected and are imputed.
3. Data constraint
 - a. The NIDA registry holds synthetic citizens, not real people.

2.5.2 Regulatory Constraints

1. Customer data must be processed according to the Tanzania Data Protection Act, 2022.
2. Data must be shared only with the customer's consent and only for the stated purpose.
3. Electronic records and transactions must comply with the Electronic Transactions Act, 2015.
4. Adverse credit information must be removable after the legal retention period; because the blockchain cannot delete, the mapping between the borrower reference and the person is deleted in PostgreSQL so the on-chain number can no longer be linked to anyone.

2.5.3 Business Constraints

1. The credit score is decision support only; each lender applies its own lending rules and a loan officer approves loans.
2. A lender must be registered in the Central System before it can push, be pulled or receive a broadcast.
3. The same borrower reference must be used by every lender for the same person.
4. Each assessment belongs to exactly one borrower and results of one borrower must never be shown to another.
5. Project duration and team size limit the number of lenders implemented (four lender systems).

2.6 User Documentation

2.6.1 Customer and Staff Documentation

1. Quick-start guide for customer registration, login with PIN and OTP, and consent management.
2. Loan officer guide for reviewing applications and requesting the Central score.

3. Central operator guide for running an assessment and broadcasting the result.
4. Credential and run-command notes kept with each system (cred.txt, run.text, connect.txt).
5. In-application messages and validation errors displayed in the user interfaces.

2.6.2 Technical Documentation

ID	Name	Description	Format / Medium
TD1	Central System README	Architecture, data model, AI model, API endpoints, security and environment variables	Markdown
TD2	API Documentation	Swagger UI at /api/docs/ and OpenAPI schema at /api/schema/ for the Central System	Online
TD3	AI Data Contract	JSON sent to and returned by the AI engine and the blockchain gateway	Markdown
TD4	Lender Subsystem Template	Contract that every lender must follow, and duplication guide	Markdown
TD5	Integration Report	NMB ↔ Central integration, payloads, tests and simulation	Markdown
TD6	Architecture and Scoring Notes	Where the score is calculated, formula, weights and bands	Markdown
TD7	Software Requirements Specification	This document	PDF, DOCX
TD8	Project Presentations	First, second and final presentations with the conceptual diagram	PDF

2.7 Assumptions and Dependencies

This section outlines the assumptions and dependencies considered critical for the successful development, deployment, and operation of DAIRE. These include technical, business, and external factors that may impact project execution.

2.7.1 Technical Assumptions

ID	Name	Description
TA1	Network Availability	1. All systems stay on the same private network. 2. IP addresses of the hosts remain reachable (or are updated in the whitelists).
TA2	Database Availability	1. PostgreSQL is installed and running for every system. 2. Database users and passwords match the Django settings.
TA3	Platform Stability	1. Python 3, Node.js and Angular tool versions in use remain compatible. 2. The Hardhat node is running when blockchain scoring is requested.
TA4	Browser Support	1. Users have modern browsers able to run the Angular interfaces.

2.7.2 Business Assumptions

ID	Name	Description
BA1	Lender Cooperation	1. Lenders agree to register in the Central System and to use the same borrower reference.

ID	Name	Description
		2. Lenders push or expose only consented data.
BA2	Regulatory Stability	1. No significant change in data protection or credit reporting rules occurs during the project.
BA3	Customer Behaviour	1. Customers are willing to give consent to share their data. 2. Customers keep their PIN private.
BA4	Demonstration Data	1. NIDA citizens and lender customers are synthetic data used for demonstration.

2.7.3 External Dependencies

ID	Name	Description
ED1	NIDA System	1. Available on port 8020 and populated with citizens. 2. Each lender's IP is in TRUSTED_CLIENT_IPS.
ED2	Central System	1. Available for lender pushes and broadcasts. 2. Lenders are registered with the correct lender ID and callback URL.
ED3	AI Model	1. The trained joblib model file is present in the Central backend. 2. An external AI engine can replace it through AI_ENGINE_URL.
ED4	Blockchain Gateway	1. The middleware and the Hardhat node are running. 2. The smart contract is deployed and its address is configured.
ED5	Third-Party Libraries	1. Django, Django REST Framework, Angular, scikit-learn, web3 and Hardhat remain compatible.

3. System Features

System features define the major services and functionalities of the DAIRE platform clearly. The sections below are the system features of the lender systems, the NIDA system, the Central System and the scoring components.

3.1 Module 1: Authentication & Identity Verification

3.1.1 Customer Registration

Field	Details
Req ID	FR-AUTH-001
Description	A new customer registers at a lender system using the NIDA number; the lender verifies the identity with the NIDA system before creating the account.
Actors	Customer, Lender System, NIDA System
Pre-condition	The customer is not yet registered and the NIDA system is reachable.
Inputs	NIDA number (20 digits), first, second and third name, region, district, ward, phone (0XXXXXXXXXX), email (optional), 4-digit PIN and confirmation
Business Rules	1. NIDA number must be exactly 20 digits and not already registered. 2. Phone number must be 10 digits starting with 0 and unique.

Field	Details
	3. PIN must be 4 digits and match its confirmation. 4. Registration is allowed only if NIDA returns matched = true.
Requirements	1. The system validates the format of all fields. 2. The lender calls NIDA POST /api/v1/nida/verify/. 3. On a match it creates the user (role customer, PIN stored as a hash). 4. It creates the customer with date of birth and gender from NIDA, is_nida_verified = true. 5. It creates the primary savings account and a pending consent. 6. It returns the account number, customer ID and masked NIDA number.
Normal Flow	1. Customer submits the registration form. 2. NIDA confirms the identity. 3. Account is created. 4. Success message with the account number is displayed.
Alternative Flow	A. NIDA number not found or names differ: HTTP 400 with the NIDA reason. B. NIDA system unreachable: HTTP 503 and the customer is asked to try later. C. Duplicate NIDA number or phone: validation error shown.
Post-condition	Customer, account and pending consent exist; the customer can set up login.
Output	Registration success message with account_number, customer_id and nida_masked.

3.1.2 NIDA Pre-Verification

Field	Details
Req ID	FR-AUTH-002
Description	The frontend checks the NIDA number and names against the NIDA registry before the customer submits the registration form.
Actors	Customer, Lender System, NIDA System
Pre-condition	The customer opened the registration page.
Inputs	NIDA number, first name, second name, third name (optional)
Business Rules	1. NIDA number, first name and second name are required. 2. The first two names must match exactly; the third name is compared only when both sides have one. 3. Names are compared without regard to case and extra spaces.
Requirements	1. The system exposes POST /api/auth/verify-nida/. 2. The system rejects malformed input before calling NIDA. 3. The system calls the NIDA client (10-second timeout). 4. The system returns matched, reason and the citizen record.
Normal Flow	1. Customer enters the NIDA number and names. 2. System returns matched = true with citizen details.

Field	Details
	3. Registration form is unlocked.
Alternative Flow	A. Not matched: HTTP 200 with matched = false and the reason. B. NIDA unreachable: HTTP 503 with a detail message.
Post-condition	The customer knows whether the identity is valid before submitting.
Output	JSON with matched, reason and citizen (name, date of birth, gender, region, district, ward).

3.1.3 User Login

Field	Details
Req ID	FR-AUTH-003
Description	A customer logs in with account number and PIN followed by a one-time password; an administrator logs in with credentials and receives JWT tokens.
Actors	Customer, Administrator, Authentication Service
Pre-condition	The user is registered and the account is active.
Inputs	Customer: account number, PIN, temp token and OTP. Administrator: login credentials.
Business Rules	1. Only active accounts can log in. 2. The same message is shown for a wrong account or a wrong PIN. 3. A new login invalidates earlier unused OTPs. 4. The OTP is valid for 10 minutes.
Requirements	1. The customer submits account number and PIN to POST /api/auth/customer-login/. 2. The system checks the account and the hashed PIN. 3. The system creates a login OTP and a temporary token. 4. The customer submits the OTP and temporary token to /api/auth/customer-login/verify-otp/. 5. The system issues an access token (12 hours) and a refresh token (7 days).

Field	Details
Normal Flow	1. Valid account number and PIN 2. OTP generated and sent to the registered phone 3. Valid OTP entered 4. Tokens issued and the dashboard opens
Alternative Flow	A. Invalid account or PIN: HTTP 401 "Invalid account number or PIN". B. OTP wrong or expired: HTTP 400 with the OTP error. C. Administrator: POST /api/auth/login/ returns the tokens directly.
Post-condition	The user is authenticated and the correct dashboard is shown (customer or admin).
Output	Access and refresh tokens and the user profile.

3.1.4 PIN Setup and Password Reset

Field	Details
Req ID	FR-AUTH-004
Description	A customer who has no PIN sets one using the account number; staff and customers can reset a password using a one-time password.
Actors	Customer, Administrator, Authentication Service
Pre-condition	The account exists; for reset the user has a registered phone number.
Inputs	Account number, new PIN and confirmation; or phone, OTP and new password
Business Rules	1. The PIN is exactly 4 digits and is stored only as a hash. 2. Reset OTPs expire after 10 minutes and can be used once. 3. Password rules of the framework apply (minimum length, not common, not numeric only).
Requirements	1. The system exposes POST /api/auth/set-pin/ and /api/auth/forgot-password/. 2. The system verifies the OTP at /api/auth/verify-otp/. 3. The system saves the new PIN or password with /api/auth/reset-password/. 4. The system confirms the change.
Normal Flow	1. User requests the change 2. OTP verified 3. New PIN or password stored
Alternative Flow	A. Account number not found: HTTP 404. B. OTP expired: user requests a new OTP.
Post-condition	The new PIN or password is active; old credentials no longer work.
Output	Success message "PIN set successfully" or password reset confirmation.

3.1.5 Profile Management

Field	Details
Req ID	FR-AUTH-005
Description	A customer views and updates the personal profile; an administrator views any customer profile.
Actors	Customer, Administrator
Pre-condition	The user is logged in.
Inputs	Profile fields (contact, employment, income, next of kin, photo)
Business Rules	1. A customer can access only their own profile. 2. NIDA number and verification status cannot be changed by the customer. 3. Every administrator update is written to the audit log. 4. The NIDA number is shown masked (first 4 and last 4 digits).

Field	Details
Requirements	1. The customer opens GET /api/customers/me/. 2. The system returns the profile with accounts and verification flags. 3. The customer submits changes with PUT/PATCH /api/customers/me/. 4. The system validates and saves the data.
Normal Flow	1. Profile displayed 2. Changes saved 3. Updated profile displayed
Alternative Flow	A. Validation error: field messages are shown. B. No profile exists: HTTP 404 "No customer profile found."
Post-condition	The profile is updated and an audit record is created for staff changes.
Output	Profile details including customer ID, masked NIDA number and verification flags.

3.2 Module 2: Accounts, Transactions & Savings

3.2.1 Account Management

Field	Details
Req ID	FR-BANK-001
Description	View accounts and their balances, and let an administrator change the account status.
Actors	Customer, Administrator
Pre-condition	The customer has at least one account (the primary savings account is created at registration).
Inputs	Account ID, new status (active, frozen, closed) and reason
Business Rules	1. Account types are savings, current and business. 2. Only active accounts can be used for login and transactions. 3. Every status change is stored in the account status log.
Requirements	1. The user opens the accounts list (/api/accounts/). 2. The system returns the customer's accounts and balances. 3. The administrator posts to update-status with the new status. 4. The system records the change and returns the status logs on request.
Normal Flow	1. Accounts listed 2. Administrator changes status 3. Log entry created
Alternative Flow	A. Invalid status value: HTTP 400. B. A customer tries another customer's account: access denied.
Post-condition	The account has the new status and the change is logged.
Output	Account list, account dashboard figures and the status log.

3.2.2 Cash Deposit

Field	Details
Req ID	FR-BANK-002
Description	Deposit money into a customer account.
Actors	Customer, Administrator, Lender System
Pre-condition	The account is active.
Inputs	Account, amount, description
Business Rules	1. Amount must be greater than zero. 2. Each transaction has a unique transaction ID and a status (pending, completed, failed, reversed). 3. Deposits update the current and available balance.
Requirements	1. The user posts to /api/transactions/deposit/. 2. The system validates the account and the amount. 3. The system records a completed transaction and updates the balance. 4. The system returns the receipt.
Normal Flow	1. Valid amount 2. Balance increased 3. Receipt returned
Alternative Flow	A. Invalid amount or inactive account: HTTP 400 with the reason.
Post-condition	Balance increased and transaction history updated.
Output	Transaction record with transaction ID, type, amount and balance after.

3.2.3 Cash Withdrawal

Field	Details
Req ID	FR-BANK-003
Description	Withdraw money from a customer account.
Actors	Customer, Administrator, Lender System
Pre-condition	The account is active and has enough balance.
Inputs	Account, amount, description
Business Rules	1. Amount must be greater than zero. 2. The available balance must cover the amount. 3. Frozen or closed accounts cannot withdraw.
Requirements	1. The user posts to /api/transactions/withdraw/. 2. The system checks status and balance. 3. The system records the withdrawal and reduces the balance. 4. The system returns the receipt.
Normal Flow	1. Sufficient balance 2. Balance reduced 3. Receipt returned

Field	Details
Alternative Flow	A. Insufficient funds: HTTP 400 "insufficient balance". B. Account frozen: request rejected.
Post-condition	Balance reduced and transaction recorded.
Output	Transaction record with transaction ID and balance after.

3.2.4 Fund Transfer

Field	Details
Req ID	FR-BANK-004
Description	Transfer money between own accounts, to another customer of the same lender, or to another institution.
Actors	Customer, Lender System
Pre-condition	The source account is active and has enough balance.
Inputs	Source account, destination (own account, customer or institution), amount
Business Rules	1. A transfer creates paired transfer-out and transfer-in records. 2. The destination must exist and be active. 3. The amount must not exceed the available balance. 4. Inter-institution transfers are recorded as a separate transfer type.
Requirements	1. The user posts to /api/transactions/transfer/ or transfer-to-customer/. 2. The system validates both accounts and the amount. 3. The system debits the source and credits the destination. 4. The system returns both transaction records.
Normal Flow	1. Valid transfer 2. Both balances updated 3. Confirmation shown
Alternative Flow	A. Destination not found: HTTP 404. B. Insufficient funds: HTTP 400.
Post-condition	Both accounts reflect the transfer in their histories.
Output	Transfer confirmation with transaction IDs.

3.2.5 Savings Management

Field	Details
Req ID	FR-BANK-005
Description	Keep a savings summary and savings records for each customer.
Actors	Customer, Administrator
Pre-condition	The customer has a savings account.
Inputs	Savings amount, date, record type
Business Rules	1. A customer sees only their own savings. 2. Savings records are kept as individual entries with a date.

Field	Details
	3. Savings data feeds the financial behaviour and the Central payload.
Requirements	1. The user opens /api/savings/. 2. The system returns the savings summary. 3. The user opens /api/savings/records/. 4. The system lists the savings records.
Normal Flow	1. Savings summary displayed 2. Records listed
Alternative Flow	A. No savings yet: empty list with a message.
Post-condition	Savings data available for reports and for the Central document.
Output	Savings summary and records.

3.2.6 Group Savings Management

Field	Details
Req ID	FR-BANK-006
Description	Manage savings and business groups, their members and their contributions.
Actors	Customer, Administrator
Pre-condition	The user is logged in.
Inputs	Group name, members, contribution amounts and dates
Business Rules	1. Only authenticated users can change group data. 2. A member belongs to a group with a role. 3. Contributions are recorded per member and per date.
Requirements	1. The user creates or opens a group (/api/groups/). 2. The user adds members (/api/groups/members/). 3. The user records contributions (/api/groups/contributions/). 4. The system lists the group totals.
Normal Flow	1. Group created 2. Members added 3. Contributions recorded
Alternative Flow	A. Member already in group: error shown.
Post-condition	The group, its members and contributions are stored.
Output	Group details with members and contributions.

3.3 Module 3: Loan Management

3.3.1 Loan Application

Field	Details
Req ID	FR-LOAN-001
Description	A customer applies for a loan; the application waits for staff review.

Field	Details
Actors	Customer, Lender System
Pre-condition	The customer is registered and has an active account.
Inputs	Loan type (personal, business, salary advance, education, agricultural), amount, term, purpose
Business Rules	1. Each application receives a unique application ID. 2. The initial status is pending. 3. Only a pending application can be cancelled by the customer.
Requirements	1. The customer posts to /api/loans/applications/. 2. The system links the application to the customer. 3. The system stores the application as pending. 4. The customer can cancel it with the cancel action while pending.
Normal Flow	1. Application submitted 2. Status pending 3. Customer sees it in the loans page
Alternative Flow	A. Invalid amount: validation error. B. Cancel on a non-pending application: HTTP 400 "Only pending applications can be cancelled."
Post-condition	The application is recorded and visible to staff.
Output	Application record with application ID and status.

3.3.2 Loan Review and Approval

Field	Details
Req ID	FR-LOAN-002
Description	A loan officer reviews an application and approves it with an interest rate and an approved amount.
Actors	Loan Officer (Administrator)
Pre-condition	The application is pending or under review.
Inputs	Application ID, interest rate, approved amount, admin notes
Business Rules	1. Only administrators can approve. 2. The default monthly interest rate is 18% and the processing fee is TZS 50,000. 3. The Central credit score is shown to the officer but does not block the decision. 4. An application already approved or disbursed cannot be approved again.
Requirements	1. The officer opens the application and optionally requests the Central score. 2. The officer posts to /api/loans/applications/{id}/approve/. 3. The system validates the rate and amount. 4. The system creates the loan and disburses it to the customer's active account.

Field	Details
	5. The system writes the audit log and notifies the customer.
Normal Flow	1. Officer approves 2. Loan created and disbursed 3. Customer notified
Alternative Flow	A. No active account: HTTP 400 "No active account found for disbursement." B. Invalid rate or amount: HTTP 400 with an error code.
Post-condition	A loan exists with status active and the customer's balance is credited.
Output	Loan record with loan ID, amount, rate and repayment plan.

3.3.3 Loan Rejection

Field	Details
Req ID	FR-LOAN-003
Description	A loan officer rejects an application and records the reason.
Actors	Loan Officer (Administrator)
Pre-condition	The application is pending or under review.
Inputs	Application ID, rejection reason
Business Rules	1. Only administrators can reject. 2. A reason must be stored with the decision. 3. A rejected application cannot be reopened.
Requirements	1. The officer posts to /api/loans/applications/{id}/reject/. 2. The system sets the status to rejected. 3. The system records the reason and the officer. 4. The system notifies the customer.
Normal Flow	1. Officer rejects 2. Status rejected 3. Customer notified
Alternative Flow	A. Application already decided: HTTP 400.
Post-condition	The application is closed with the reason.
Output	Updated application with status rejected.

3.3.4 Repayment Schedule and Payment

Field	Details
Req ID	FR-LOAN-004
Description	Show the repayment schedule of a loan and record payments of installments.
Actors	Customer, Loan Officer, Lender System
Pre-condition	The loan is active.

Field	Details
Inputs	Loan ID, installment, amount paid
Business Rules	1. Repayment frequency is monthly, biweekly or weekly. 2. Loan status can be pending, active, completed, overdue, defaulted or cancelled. 3. A late or missed payment increases overdue days and may change the loan status. 4. Paying all installments completes the loan.
Requirements	1. The user opens /api/loans/{id}/schedule/. 2. The system lists the installments with due dates. 3. The user or officer marks an installment as paid (/api/repayments/{id}/mark-paid/). 4. The system updates balances and the repayment record.
Normal Flow	1. Schedule displayed 2. Installment paid 3. Outstanding balance reduced
Alternative Flow	A. Installment already paid: error shown.
Post-condition	Repayment history reflects the payment; loan may become completed.
Output	Repayment record and updated loan balance.

3.3.5 Credit History and Inquiries

Field	Details
Req ID	FR-LOAN-005
Description	Keep the credit history, credit inquiries and financial behaviour of each customer.
Actors	Customer, Loan Officer
Pre-condition	The customer has a profile.
Inputs	Credit history data, inquiry details
Business Rules	1. A customer sees only their own history. 2. Inquiries record who requested the information and when. 3. Financial behaviour indicators are calculated from transactions and repayments.
Requirements	1. The user opens /api/credit-history/. 2. The system returns the credit history summary. 3. The user opens /api/credit-history/inquiries/ and /api/financial-behaviour/. 4. The system returns the inquiries and indicators.
Normal Flow	1. History displayed 2. Inquiries and behaviour displayed
Alternative Flow	A. No data: empty state shown.

Field	Details
Post-condition	The data is available to officers and to the Central payload.
Output	Credit history, inquiries and financial behaviour records.

3.4 Module 4: Consent & Data Sharing

3.4.1 Grant Consent

Field	Details
Req ID	FR-CONS-001
Description	A customer grants consent to share selected data categories with the Central System.
Actors	Customer
Pre-condition	The customer is logged in; a pending consent exists from registration.
Inputs	Consent ID, duration in days (default 365), shared data categories
Business Rules	1. No data may leave the lender without an active consent. 2. Consent has a status: pending, active, revoked or expired. 3. The consent date, expiry date and IP address are recorded.
Requirements	1. The customer opens the consent page. 2. The customer chooses the categories and the duration. 3. The customer posts to /api/consent/{id}/grant/. 4. The system sets the status active and writes the audit log CONSENT_GRANTED.
Normal Flow	1. Consent granted 2. Status active 3. Expiry date shown
Alternative Flow	A. Consent not found or not owned by the customer: access denied.
Post-condition	The consent is active until its expiry date.
Output	Consent record with status, granted date and expiry date.

3.4.2 Revoke and Extend Consent

Field	Details
Req ID	FR-CONS-002
Description	A customer revokes an active consent or extends its validity.
Actors	Customer
Pre-condition	The consent exists.
Inputs	Consent ID, extra days for extension
Business Rules	1. Revoking stops all future pushes to Central. 2. The revocation date is recorded. 3. An expired consent must be renewed before data can be shared.

Field	Details
Requirements	1. The customer posts to /api/consent/{id}/revoke/ or /extend/. 2. The system updates the status and the dates. 3. The system writes an audit log. 4. The system returns the updated consent.
Normal Flow	1. Consent revoked or extended 2. Audit log written
Alternative Flow	A. Extension of a revoked consent: rejected.
Post-condition	The consent has the new status or the new expiry date.
Output	Updated consent record.

3.4.3 Consent-Based Data Category Enforcement

Field	Details
Req ID	FR-CONS-003
Description	Before a push to Central, the system includes only the data categories allowed by the customer's consent.
Actors	Lender System, Loan Officer
Pre-condition	An active consent exists.
Inputs	Customer ID and requested categories
Business Rules	1. Allowed categories: profile, transactions, savings, loans, repayments, credit_history, financial_behaviour. 2. Bank-only data (NIDA number, phone, email, address, next of kin) is never sent. 3. Each category has a fixed list of consented fields.
Requirements	1. The officer requests a push for chosen categories. 2. The system checks the consent status and the categories. 3. The system builds the flat Central document with the permitted fields. 4. The system rejects any category that is not consented.
Normal Flow	1. Consent verified 2. Payload built 3. Push continues
Alternative Flow	A. No active consent: HTTP 403 and nothing is sent. B. Unknown category: HTTP 400.
Post-condition	Only permitted data is in the payload.
Output	Filtered Central document or a consent error.

3.5 Module 5: Lender–Central Integration

3.5.1 Push Customer Data to Central

Field	Details
Req ID	FR-INTG-001
Description	A loan officer pushes a customer’s consented data to the Central System.
Actors	Loan Officer, Lender System, Central System
Pre-condition	Active consent; the lender is registered in Central.
Inputs	Customer ID and categories
Business Rules	1. Headers identify the lender (X-Lender-Id, X-Source-System, X-Source-Version). 2. Only administrators can push (POST /api/daire/central/push/). 3. Every push is stored in the data sync log with its status.
Requirements	1. The officer selects the customer and categories. 2. The system builds the payload and checks consent. 3. The system posts it to the Central receiver /api/lender-data/receive/. 4. The system stores the result in the sync log.
Normal Flow	1. Push requested 2. Central accepts 3. Sync log marked successful
Alternative Flow	A. Central unreachable: log marked failed and available for retry. B. No consent: push refused.
Post-condition	Central holds the customer’s data under the borrower reference.
Output	Sync record with status and Central response.

3.5.2 Receive Central Assessment Result

Field	Details
Req ID	FR-INTG-002
Description	The lender receives the broadcast credit result from Central and stores it for the matching customer.
Actors	Central System, Lender System
Pre-condition	The lender is registered in Central and its callback URL is configured.
Inputs	Result JSON with NIDA number, credit score, risk band, dimensions, AI fields, assessment reference
Business Rules	1. The callback accepts only trusted source IPs (TRUSTED_CALLBACK_IPS) or an administrator. 2. A broadcast result must contain the NIDA number and credit score. 3. The lender keeps only its own entry from a multi-lender broadcast.
Requirements	1. Central posts to /api/daire/central/receive/. 2. The system checks the source IP.

Field	Details
	3. The system matches the customer by the identifiers sent. 4. The system stores the result as a Central record.
Normal Flow	1. Result received 2. Customer matched 3. Record stored
Alternative Flow	A. Source IP not trusted: HTTP 403. B. No matching customer: result rejected with a message.
Post-condition	The customer and the loan officer can see the score.
Output	Stored Central record linked to the customer.

3.5.3 Central Borrower Lookup

Field	Details
Req ID	FR-INTG-003
Description	Central pulls a borrower's permitted data from the lender using the borrower reference.
Actors	Central System, Lender System
Pre-condition	The lender is CONNECTED or DEGRADED in Central.
Inputs	borrower_reference
Business Rules	1. Only trusted IPs may call the endpoint. 2. The lender returns only data the customer consented to share. 3. A not-found response is returned when the customer does not exist; no other customer's data is returned.
Requirements	1. Central calls GET /api/daire/borrowers/?borrower_reference=... 2. The system finds the customer. 3. The system builds the permitted payload. 4. The system returns it as JSON.
Normal Flow	1. Central requests a borrower 2. Lender returns the payload
Alternative Flow	A. Customer not found: HTTP 404. B. Untrusted IP: HTTP 403.
Post-condition	Central can merge the borrower's data from this lender.
Output	Borrower payload for the Central exchange.

3.5.4 Synchronisation Logs and Retry

Field	Details
Req ID	FR-INTG-004
Description	Administrators list synchronisation logs and retry failed or pending pushes.
Actors	Loan Officer (Administrator)

Field	Details
Pre-condition	At least one push has been attempted.
Inputs	Sync ID
Business Rules	1. Only administrators can see logs and retry. 2. Only failed or pending pushes can be retried. 3. Each retry updates the same log with the new outcome.
Requirements	1. The administrator opens /api/daire/central/sync-logs/. 2. The system lists logs with status and time. 3. The administrator posts to /sync-logs/{id}/retry/. 4. The system repeats the push and updates the log.
Normal Flow	1. Logs displayed 2. Retry executed 3. Status updated
Alternative Flow	A. Log not eligible: HTTP 400.
Post-condition	The log shows the final status.
Output	List of sync logs and retry result.

3.5.5 View Central Credit Score

Field	Details
Req ID	FR-INTG-005
Description	Customers see their received credit result; loan officers request the Central score for an application.
Actors	Customer, Loan Officer
Pre-condition	A Central result has been received, or the officer requests one.
Inputs	Application ID and categories (officer); none (customer)
Business Rules	1. A customer can read only their own Central records (/api/daire/central/my-data/). 2. The officer's request uses /api/loans/applications/{id}/central-score/. 3. The score is informational and does not decide the loan automatically.
Requirements	1. The customer opens the credit score page. 2. The system returns the latest result. 3. The officer requests the score for an application. 4. The system pushes data to Central and returns the stored result.
Normal Flow	1. Customer opens page 2. Score and risk level displayed
Alternative Flow	A. No result yet: message that no score has been received.
Post-condition	The decision maker sees the score.
Output	Credit score, risk band, dimensions, AI reputation and behaviour summary.

3.6 Module 6: Central System Operations

3.6.1 Lender Registration and Status

Field	Details
Req ID	FR-CTR-001
Description	Register lenders in the Central System and keep their connection status.
Actors	Central Operator
Pre-condition	The operator is logged in.
Inputs	lender_id, institution name, institution type, API base URL, status
Business Rules	1. lender_id is unique. 2. Status is CONNECTED, DEGRADED or DISCONNECTED. 3. Only CONNECTED and DEGRADED lenders are pulled. 4. Registered lenders: CRDB-01, MoFinance-004, NMB-01 and NMB-02.
Requirements	1. The operator posts to /api/lenders/. 2. The system stores the lender. 3. The operator updates it with PATCH /api/lenders/{id}/. 4. The console shows the network status.
Normal Flow	1. Lender registered 2. Status shown as CONNECTED
Alternative Flow	A. Duplicate lender_id: validation error.
Post-condition	The lender can push data and receive broadcasts.
Output	Lender record and status.

3.6.2 Receive and Merge Lender Data

Field	Details
Req ID	FR-CTR-002
Description	Receive lender data and merge it into the borrower identified by the borrower reference.
Actors	Lender System, Central System
Pre-condition	The lender is registered.
Inputs	lender_id, borrower_reference, payload (accounts, loans, repayments, profile)
Business Rules	1. borrower_reference is the single identity used to match data. 2. Data is merged only into the requested borrower, never across borrowers. 3. Each record keeps its source lender; differences are stored as conflicts. 4. The lender can be resolved from the body, headers or API key.
Requirements	1. The lender posts to /api/lender-data/receive/. 2. The system validates the payload. 3. The system merges accounts, loans, repayments and the financial

Field	Details
	profile. 4. The system recalculates the borrower aggregates and records the exchange.
Normal Flow	1. Data received 2. Merged by borrower reference 3. Aggregates recalculated
Alternative Flow	A. Lender unknown: rejected. B. Invalid payload: HTTP 400 with details.
Post-condition	The borrower has an up-to-date unified record.
Output	Acceptance response with the borrower reference.

3.6.3 Pull Borrower Data from All Lenders

Field	Details
Req ID	FR-CTR-003
Description	Central pulls one borrower's data concurrently from all eligible lenders.
Actors	Central Operator
Pre-condition	Lenders are registered and CONNECTED or DEGRADED.
Inputs	borrower_reference
Business Rules	1. A lender slower than the lender timeout (8 seconds) fails independently. 2. Each response must contain the requested borrower reference. 3. DISCONNECTED lenders are skipped.
Requirements	1. The operator posts to /api/borrowers/pull-from-all-lenders/. 2. The system calls every eligible lender at the same time. 3. The system validates and merges each successful response. 4. The system reports success and failure per lender.
Normal Flow	1. Pull requested 2. Responses merged 3. Per-lender result shown
Alternative Flow	A. One lender fails: other lenders are still used.
Post-condition	The borrower record contains data from all reachable lenders.
Output	Per-lender pull results.

3.6.4 Data Routing Policy

Field	Details
Req ID	FR-CTR-004
Description	Control which normalized fields may leave Central towards lenders, AI and blockchain.
Actors	Central Operator

Field	Details
Pre-condition	The operator is logged in.
Inputs	Field lists for lenders, AI and blockchain
Business Rules	1. Only fields allowed by the active policy are sent to each destination. 2. Raw lender data stays in Central. 3. The blockchain receives only dimensions, never identity data.
Requirements	1. The operator opens /api/routing-policies/. 2. The system returns the active policy. 3. The operator updates the field lists. 4. The system applies the policy to later exchanges.
Normal Flow	1. Policy displayed 2. Policy updated
Alternative Flow	A. Unknown field name: validation error.
Post-condition	Later exchanges use the new policy.
Output	Policy record with the three field lists.

3.6.5 Data Exchange Audit

Field	Details
Req ID	FR-CTR-005
Description	Record and display every inter-system operation.
Actors	Central Operator, Auditor
Pre-condition	Exchanges have taken place.
Inputs	Filters (system, direction, status, borrower, lender)
Business Rules	1. Each exchange stores system, direction, operation, status, borrower, lender, assessment, fields sent, payload, response and error message. 2. Records are read-only.
Requirements	1. The operator opens /api/data-exchanges/. 2. The system lists the exchanges. 3. The operator filters by status or lender. 4. The system shows details and errors.
Normal Flow	1. Exchanges listed 2. Details opened
Alternative Flow	A. No exchanges: empty list.
Post-condition	The history supports audit and troubleshooting.
Output	List of data exchange records.

3.6.6 Operations Dashboard and Network Status

Field	Details
Req ID	FR-CTR-006

Field	Details
Description	Show dashboard figures and the health of every registered lender link.
Actors	Central Operator
Pre-condition	The operator is logged in.
Inputs	None (refresh action)
Business Rules	1. Each lender shows its status and its number of exchanges. 2. The console shows when each lender was last active. 3. Aggregates come from /api/dashboard/.
Requirements	1. The operator opens the Data exchange page. 2. The system draws the network with the Central node and each lender. 3. The system lists registered lenders with status and exchange count. 4. The operator refreshes the data.
Normal Flow	1. Network status displayed 2. Refresh updates the figures
Alternative Flow	A. A lender is disconnected: it is shown as disconnected.
Post-condition	The operator knows which lenders are connected.
Output	Dashboard figures and network status view.

3.7 Module 7: AI Reputation & Blockchain Scoring

3.7.1 Create Assessment

Field	Details
Req ID	FR-SCR-001
Description	Create one assessment for one borrower and build the normalized credit features.
Actors	Central Operator
Pre-condition	The borrower has merged data in Central.
Inputs	borrower reference
Business Rules	1. One assessment belongs to one borrower and has a unique assessment reference. 2. Eleven normalized features are built: loan counts, debt, on-time ratio, missed and late payments, maximum days overdue, transaction and income frequency and balance stability. 3. Five dimensions D1–D5 (0–100) are derived for the blockchain.
Requirements	1. The operator posts to /api/assessments/. 2. The system builds the credit profile and features. 3. The system derives the dimensions. 4. The system returns the assessment reference.
Normal Flow	1. Assessment created 2. Features and dimensions stored

Field	Details
Alternative Flow	A. Borrower without data: error shown.
Post-condition	The assessment is ready for AI and blockchain scoring.
Output	Assessment record with reference, features and dimensions.

3.7.2 AI Reputation Analysis

Field	Details
Req ID	FR-SCR-002
Description	Run the AI model to obtain the default probability, reputation and risk level.
Actors	Central Operator, AI Engine
Pre-condition	An assessment exists.
Inputs	11 normalized features
Business Rules	1. The local model is a Random Forest pipeline (100 trees); an external engine can be used through AI_ENGINE_URL. 2. Reputation score = 1 – default probability. 3. Score ≥ 0.85 EXCELLENT/LOW, ≥ 0.70 GOOD/LOW, ≥ 0.55 MODERATE/MEDIUM, otherwise HIGH_RISK/HIGH. 4. If the model file cannot be loaded, a deterministic fallback is used.
Requirements	1. The operator posts to /api/assessments/{reference}/ai-reputation/. 2. The system maps the borrower to the model input. 3. The system runs the model and stores the result. 4. The result is read at /ai-result/.
Normal Flow	1. AI requested 2. Reputation and risk stored
Alternative Flow	A. External AI timeout: error recorded and the fallback may be used.
Post-condition	The assessment contains the AI result.
Output	Reputation, score, risk level, behaviour summary and model version.

3.7.3 Blockchain Credit Score Calculation

Field	Details
Req ID	FR-SCR-003
Description	Send the five dimensions to the blockchain and obtain the credit score and transaction hash.
Actors	Central Operator, Middleware, Smart Contract
Pre-condition	An assessment exists with dimensions.
Inputs	D1–D5 dimensions and assessment reference
Business Rules	1. No personal data is sent; only derived numbers. 2. Weights: D1 30%, D2 20%, D3 15%, D4 20%, D5 15%.

Field	Details
	3. Score = $300 + (\text{weighted sum} \times 550) / 10,000$, range 300–850, FICO risk bands. 4. A dimension below 40 limits the band to FAIR; insufficient evidence gives band NONE.
Requirements	1. The operator posts to <code>/api/assessments/{reference}/blockchain-score/</code> . 2. Central calls the middleware, which calls the contract. 3. The contract calculates and stores the score. 4. Central stores the score, hash, network and block number.
Normal Flow	1. Blockchain scoring requested 2. Score and hash stored 3. Transaction verifiable at <code>/verify/</code>
Alternative Flow	A. Gateway unavailable: error recorded; the operator can retry.
Post-condition	The assessment contains the credit score.
Output	Credit score, risk band, transaction hash and ruleset version.

3.7.4 Broadcast Assessment Result

Field	Details
Req ID	FR-SCR-004
Description	Broadcast the same assessment result to every lender linked to the borrower.
Actors	Central Operator, Lender Systems
Pre-condition	AI and blockchain results exist for the assessment.
Inputs	Borrower ID
Business Rules	1. The broadcast contains only the result of that borrower and that assessment. 2. Every lender linked to the borrower receives the same score and analysis. 3. Each lender applies its own lending rules. 4. The operator triggers the broadcast after reviewing the result.
Requirements	1. The operator posts to <code>/api/borrowers/{id}/broadcast-result/</code> . 2. The system finds the linked lenders. 3. The system posts the result to each lender's <code>/api/daire/central/receive/</code> . 4. The system records each delivery in the data exchange log.
Normal Flow	1. Broadcast triggered 2. Lenders receive the result 3. Deliveries logged
Alternative Flow	A. A lender is unreachable: delivery marked failed; others continue.
Post-condition	All linked lenders hold the same result.
Output	Delivery status per lender.

3.8 Module 8: NIDA Registry, Reporting & Administration

3.8.1 NIDA Identity Verification Service

Field	Details
Req ID	FR-NIDA-001
Description	The NIDA system verifies a NIDA number and names for a trusted lender.
Actors	Lender System, NIDA System
Pre-condition	The caller IP is in TRUSTED_CLIENT_IPS and the citizen is active.
Inputs	nida_number, first_name, second_name, third_name
Business Rules	1. NIDA number is 20 digits (YYYYMMDD plus 12 digits). 2. Only active citizens are verified. 3. Names are compared without case and extra spaces; the third name is optional. 4. A failed match still returns HTTP 200 with matched = false.
Requirements	1. The lender posts to /api/v1/nida/verify/. 2. NIDA finds the active citizen. 3. NIDA compares the names. 4. NIDA returns matched, reason and the citizen record.
Normal Flow	1. Request received 2. Identity matched 3. Citizen record returned
Alternative Flow	A. "NIDA number not found in the national registry." B. "Name does not match the NIDA record."
Post-condition	The lender can accept or refuse the registration.
Output	JSON: matched, reason, citizen.

3.8.2 NIDA Lookup, Health and Registry Listing

Field	Details
Req ID	FR-NIDA-002
Description	Look up a citizen by NIDA number, check the service health and list citizens.
Actors	Administrator, Lender System, NIDA System
Pre-condition	The caller IP is trusted (health is open).
Inputs	nida_number (lookup)
Business Rules	1. Lookup requires nida_number and returns 404 when not found. 2. Health returns the status, service name and the number of citizens. 3. The citizens list is read-only and paginated (50 per page). 4. Citizens are synthetic: 200 loaded from the dump or the fixture.
Requirements	1. The caller requests /lookup/?nida_number=... 2. NIDA returns the full record.

Field	Details
	3. The caller requests /health/ and /citizens/count/. 4. NIDA returns the status and counts.
Normal Flow	1. Lookup returns a citizen 2. Health shows status ok
Alternative Flow	A. Missing parameter: HTTP 400. B. Unknown number: HTTP 404.
Post-condition	Operators know that NIDA is running with data.
Output	Citizen record, health status and counts.

3.8.3 Reports and Analytics

Field	Details
Req ID	FR-ADM-001
Description	Administrators view reports on customers, money flows, loans and risk.
Actors	Loan Officer (Administrator)
Pre-condition	The administrator is logged in.
Inputs	Date range and filters
Business Rules	1. Reports: overview, customers, transactions, savings, loans, repayments, defaults, credit history and financial behaviour. 2. The overview shows totals such as customers, loans issued, deposits, outstanding and default amounts. 3. Filters are validated before use.
Requirements	1. The administrator opens the reports page. 2. The system validates the filters. 3. The system calculates the figures. 4. The system displays tables and totals.
Normal Flow	1. Report selected 2. Figures displayed
Alternative Flow	A. Invalid filter: HTTP 400 with the reason.
Post-condition	The administrator has the figures needed for decisions.
Output	Report data in JSON for display.

3.8.4 Audit Logs and Notifications

Field	Details
Req ID	FR-ADM-002
Description	Record important actions in an audit log and send notifications to users.
Actors	Administrator, Customer
Pre-condition	The user is logged in.
Inputs	Filters; notification ID

Field	Details
Business Rules	1. Audit entries store user, action, module, record ID, IP address, description and customer. 2. Only administrators can read the audit logs. 3. Users can mark notifications as read (single or all).
Requirements	1. The system writes an audit entry for decisions, consent and updates. 2. The administrator lists /api/audit/logs/. 3. The user opens /api/notifications/. 4. The user marks notifications as read.
Normal Flow	1. Audit logs listed 2. Notifications read
Alternative Flow	A. No entries: empty list.
Post-condition	Actions are traceable and notifications are cleared.
Output	Audit log list and notification list.

3.8.5 System Settings and Lender Credentials

Field	Details
Req ID	FR-ADM-003
Description	Administrators manage runtime settings and the lender registry used for the Central integration.
Actors	Loan Officer (Administrator)
Pre-condition	The administrator is logged in.
Inputs	Setting key and value; lender credential fields
Business Rules	1. Only administrators can change settings. 2. Secrets such as API keys are never shown in clear text or logs. 3. Central URL, lender ID and callback URL are configured here or in the environment file.
Requirements	1. The administrator opens /api/system/settings/. 2. The system lists the settings. 3. The administrator updates a value. 4. The system saves it and applies it on the next call.
Normal Flow	1. Settings displayed 2. Value updated
Alternative Flow	A. Invalid value: validation error.
Post-condition	The system uses the new configuration.
Output	Settings and lender credential records.

4. External Interface Requirements

4.1 User Interfaces

All user interfaces are web-based Angular applications. Each lender system has its own user interface built from the same lender template, so the four lender systems have different looks but the same functions.

4.1.1 Authentication Screens (all lender systems)

1. Login: account number and PIN, then one-time password; administrator login.
2. Register: NIDA number, names, location, phone, email and PIN, with NIDA pre-verification.
3. Forgot password and reset password.

4.1.2 Customer Dashboard

1. Overview with balances and recent activity.
2. Accounts, transactions (deposit, withdraw, transfer), savings and repayments.
3. Loans: apply, view applications and the repayment schedule.
4. Consent: grant, revoke and extend.
5. Credit history, financial behaviour and the DAIRE credit score page.
6. Notifications and profile.

4.1.3 Lender Administrator Dashboard

1. Overview with key figures.
2. Customers (list and detail), accounts, transactions, savings, groups.
3. Loan applications (review, approve, reject), loans and repayments.
4. Consent, credit history and audit logs.
5. DAIRE page for pushing data to Central, Central scores and AI analysis.
6. Reports, notifications and settings.

4.1.4 Central System Operations Console

1. Overview: dashboard aggregates.
2. Data exchange: network status with the Central node and each lender, and the list of registered lenders with status and exchange count (for example CRDB-01, MoFinance-004, NMB-01, NMB-02).
3. Data management: lenders, borrowers, assessments, routing policy and broadcast.
4. Log out.

4.1.5 Common Interface Requirements

1. Validation messages are shown next to the field that caused the error.
2. Confirmation is requested before decisions that cannot be undone (approve, reject, revoke).
3. The NIDA number is displayed masked.

4.2 Hardware Interfaces

DAIRE is a web-based software system and has no direct hardware interface. The following hardware characteristics apply:

1. Server hosts: Linux computers on the local network running the Django servers, PostgreSQL, the middleware and the Hardhat node.
2. Client devices: computers, tablets and phones with a modern web browser.
3. Network equipment: wired and wireless local network with fixed or reserved addresses for the servers.

4. No biometric device, card reader, printer or point-of-sale device is required.

4.3 Software Interfaces

ID	Interface	Description
SI-01	Lender → NIDA	POST /api/v1/nida/verify/, GET /lookup/, GET /health/ over HTTP/JSON. Request: NIDA number and names; response: matched, reason, citizen. Timeout 10 seconds; the client returns an unavailable result on errors.
SI-02	Lender → Central	POST /api/lender-data/receive/ with the flat customer document. Headers: X-Lender-Id, X-Source-System, X-Source-Version.
SI-03	Central → Lender	POST {api_base_url}/api/daire/central/receive/ with the credit result. GET /api/daire/borrowers/?borrower_reference=... for pulls.
SI-04	Central → AI	Local joblib Random Forest model or external endpoint AI_ENGINE_URL. Input: 11 normalized features; output: reputation, score, risk, summary.
SI-05	Central → Blockchain	Middleware (port 5000): POST /api/v1/score/submit and /submit-calculated, GET /api/v1/score/{borrowerRef}. Smart contract DaireCreditScore on the Hardhat network.
SI-06	Application → Database	PostgreSQL through the Django ORM (psycopg2).

4.4 Communication Interfaces

ID	Item	Description
CI-01	Protocol	HTTP with JSON bodies (Content-Type: application/json) between all systems; HTTPS or VPN is required outside the trusted network.
CI-02	Ports	NIDA 8020; lender backends 8002; Central backend 8000; middleware 5000; blockchain 8545; Central Postgres 5434; interfaces 4200 (Central), 4300 (NMB), 4600 (CRDB).
CI-03	User authentication	Authorization: Bearer <access token> (JWT) for users.
CI-04	System authentication	Trusted source IP lists: TRUSTED_CLIENT_IPS (NIDA) and TRUSTED_CALLBACK_IPS (lenders); optional X-API-Key when REQUIRE_API_KEYS=true in Central.
CI-05	Error format	Business failures return HTTP 200 with matched = false (NIDA); other errors return 400, 401, 403, 404 or 503 with a detail message.
CI-06	Timeouts	NIDA 10 s; NIDA health 3 s; lender pull 8 s; AI and blockchain 30 s.

5. Non-Functional Requirements

5.1 Performance Requirements

5.1.1 Response Time Requirements

ID	Title	Requirement Details
NFR-PERF-001	API Response	1. Ordinary lender API calls shall respond within 3 seconds for 95% of requests on the local network. 2. Login steps shall complete within 3 seconds.
NFR-PERF-002	NIDA Verification	1. A NIDA verification shall respond within 2 seconds when the NIDA system is available. 2. The client shall give up after 10 seconds and report that NIDA is unavailable.
NFR-PERF-003	Scoring	1. AI scoring shall complete within 30 seconds. 2. Blockchain scoring shall complete within 30 seconds. 3. A lender pull shall not wait more than 8 seconds for one lender.

5.1.2 Throughput Requirements

ID	Title	Requirement Details
NFR-PERF-004	Concurrent Users	1. Each lender system shall support at least 50 concurrent users. 2. The Central System shall support at least 10 concurrent operators.
NFR-PERF-005	Batch Operations	1. A pull from all lenders shall run lender calls concurrently. 2. Lists shall be paginated (20 items in Central, 50 in NIDA).

5.1.3 Resource Utilization

ID	Title	Requirement Details
NFR-PERF-006	Server Resources	1. A lender backend shall run on a computer with 4 GB of memory. 2. Database connections shall be reused through the framework.
NFR-PERF-007	Client Resources	1. The Angular interface shall load on a standard browser without additional plugins.

5.1.4 Scalability Requirements

ID	Title	Requirement Details
NFR-SCAL-001	More Lenders	1. A new lender shall be added by duplicating the lender template and registering it in Central, without changing Central code. 2. Central shall pull and broadcast to all registered lenders.
NFR-SCAL-002	Data Growth	1. The databases shall be PostgreSQL to support growth in customers, transactions and loans. 2. Central shall accept compact aggregates instead of raw transaction rows.
NFR-SCAL-003	Separation	1. NIDA, lender, Central, middleware and AI components shall be separate processes that can run on separate computers.

5.2 Security Requirements

5.2.1 Authentication and Authorization

ID	Title	Requirement Details
NFR-SEC-001	User Authentication	1. Customers shall log in with account number, PIN and OTP. 2. The PIN shall be stored only as a hash. 3. The OTP shall expire after 10 minutes and be used once. 4. Access tokens shall expire after 12 hours and refresh tokens after 7 days.
NFR-SEC-002	Role-Based Access	1. Customers shall access only their own data. 2. Loan decisions, reports, audit logs and settings shall be limited to administrators.
NFR-SEC-003	System Authentication	1. NIDA shall accept requests only from IPs in TRUSTED_CLIENT_IPS. 2. Lenders shall accept Central callbacks only from IPs in TRUSTED_CALLBACK_IPS. 3. Where a system is exposed beyond the trusted network, an API key shall be required.

5.2.2 Data Protection

ID	Title	Requirement Details
NFR-SEC-004	Data Minimization	1. Only consented categories shall be sent to Central. 2. NIDA number, phone, email, address and next of kin shall not be sent in the Central document. 3. Only derived dimensions shall be sent to the blockchain.
NFR-SEC-005	Secrets	1. Passwords, PINs and API keys shall not be written in logs or source control. 2. Credentials shall be kept in environment files.
NFR-SEC-006	Transport	1. Traffic outside the trusted network shall use HTTPS or a VPN. 2. Inbound API keys in Central shall be stored as hashes.

5.2.3 Privacy and Compliance

ID	Title	Requirement Details
NFR-SEC-007	Privacy Law	1. Personal data shall be processed according to the Tanzania Data Protection Act, 2022. 2. The customer shall be able to revoke consent at any time.
NFR-SEC-008	On-Chain Privacy	1. No name, NIDA number or phone number shall be written on-chain. 2. The borrower reference on-chain shall be a hash that cannot be reversed to the person.
NFR-SEC-009	Erasure	1. Deleting the mapping between the borrower reference and the person in PostgreSQL shall make the on-chain number unlinkable (cryptographic erasure).

5.2.4 Security Monitoring and Incident Response

ID	Title	Requirement Details
NFR-SEC-010	Audit Trail	1. Decisions, consent changes, profile updates and pushes to Central shall be written to the audit log. 2. Every inter-system exchange shall be written to the data exchange log.
NFR-SEC-011	Detection	1. Failed integrations shall appear with status failed in the sync logs and the data exchange log. 2. Repeated rejected requests from an untrusted IP shall be visible in the server log.
NFR-SEC-012	Response	1. An exposed credential shall be revoked and replaced, and the environment files updated.

5.3 Reliability and Availability

5.3.1 Availability Requirements

ID	Title	Requirement Details
NFR-REL-001	Availability	1. Each system shall be available during the demonstration and operating hours. 2. Start and stop scripts shall restart the services in the correct order.

5.3.2 Fault Tolerance

ID	Title	Requirement Details
NFR-REL-002	Partial Failure	1. If one lender is slow or offline, an assessment shall continue with the other lenders. 2. If NIDA is unreachable, registration shall return a clear "unavailable" message and create nothing.
NFR-REL-003	AI Fallback	1. If the AI model file cannot be loaded, a local fallback formula shall be used and marked with its version.

5.3.3 Disaster Recovery

ID	Title	Requirement Details
NFR-REL-004	Backup	1. Each PostgreSQL database shall be backed up with pg_dump (for example nida_registry and the lender databases). 2. Backups shall be kept outside the application folder.
NFR-REL-005	Restore	1. A restore shall use pg_restore and the Django migrations shall be consistent with the restored data. 2. The NIDA registry can be restored from its dump file or reseeded from the citizen fixture.

5.3.4 Error Handling and Recovery

ID	Title	Requirement Details
NFR-REL-006	Clear Errors	1. Errors shall be returned with a status code and a readable message. 2. Validation errors shall name the field.

ID	Title	Requirement Details
NFR-REL-007	Retry	1. Failed or pending pushes shall be retried from the sync log. 2. Failed assessments shall be re-run without creating a duplicate borrower.
NFR-REL-008	Consistency	1. Registration shall run inside a database transaction so that user, customer and account are all created or none is created.

5.4 Usability and Accessibility

5.4.1 User Interface Usability

ID	Title	Requirement Details
NFR-USE-001	Simplicity	1. Core tasks (register, log in, apply for a loan, grant consent) shall be reachable within three clicks from the dashboard. 2. Forms shall show inline validation.

5.4.2 Accessibility Standards

ID	Title	Requirement Details
NFR-USE-002	Accessibility	1. Text shall have sufficient contrast and readable size. 2. Forms shall have labels and be usable with the keyboard.

5.4.3 Multilingual Support

ID	Title	Requirement Details
NFR-USE-003	Language	1. The interface shall be in English. 2. Text shall be stored in UTF-8 so Swahili names and places display correctly.
NFR-USE-004	Local Formats	1. Amounts shall be shown in Tanzanian shillings (TZS). 2. Dates and times shall use the Africa/Dar_es_Salaam time zone.

5.4.4 User Experience Optimization

ID	Title	Requirement Details
NFR-USE-005	Feedback	1. Long operations (assessment, pull, broadcast) shall show a progress or result message. 2. Success and failure shall be shown clearly.
NFR-USE-006	Role Views	1. Customers and administrators shall see different menus according to their role.

5.5 Maintainability and Portability

5.5.1 Maintainability Requirements

ID	Title	Requirement Details
NFR-MAIN-001	Modular Code	1. Each business area shall be a separate Django app (accounts, customers, loans, consent, daire_integration and others). 2. The lender template shall be duplicated for each new lender.
NFR-MAIN-002	Tests	1. Automated tests shall cover verification, registration, integration and transactions (for example the 19 integration tests of NMB).

ID	Title	Requirement Details
		2. Tests shall be run with python manage.py test.

5.5.2 Portability Requirements

ID	Title	Requirement Details
NFR-PORT-001	Environment	1. Configuration shall be read from environment files and settings, not hard coded in views. 2. The systems shall run on Linux with Python virtual environments.

5.5.3 Compatibility Requirements

ID	Title	Requirement Details
NFR-COMP-001	Software Versions	1. NIDA and lenders: Python 3 with Django 4.2.16, Django REST Framework 3.15.2, PostgreSQL. 2. Central: Django 5.2 with drf-spectacular. 3. Interfaces: Angular; middleware: Node.js.
NFR-COMP-002	Data Contracts	1. Lenders shall follow the lender JSON contract (borrower reference, accounts, transactions, loans, repayments). 2. The AI input shall have the 11 agreed fields.

5.5.3 Compatibility Requirements (continued)

ID	Title	Requirement Details
NFR-COMP-003	Identity Key	1. The NIDA number (20 digits) shall be the common identity across all systems. 2. Central shall use one borrower reference per person.
NFR-COMP-004	Browsers	1. The interfaces shall run in current versions of Chrome, Firefox and Edge.
NFR-COMP-005	Interchangeability	1. Another AI engine or another blockchain gateway shall be usable by changing AI_ENGINE_URL or BLOCKCHAIN_RPC_URL.

5.6 Legal and Compliance Requirements

5.6.1 Regulatory Compliance

ID	Title	Requirement Details
NFR-LEG-001	Data Protection	1. The system shall comply with the Tanzania Data Protection Act, 2022. 2. Data shall be shared only with a valid consent.
NFR-LEG-002	Electronic Records	1. Electronic records and transactions shall comply with the Electronic Transactions Act, 2015.
NFR-LEG-003	Retention	1. Adverse information shall be removable after the legal retention period through cryptographic erasure.

5.6.2 Intellectual Property and Licensing

ID	Title	Requirement Details
NFR-LEG-004	Ownership	1. The source code and documents belong to the project owners

ID	Title	Requirement Details
		under the e-GovRIDC programme.
NFR-LEG-005	Open Source	1. Open-source components (Django, Angular, scikit-learn, Hardhat and others) shall be used according to their licences.

5.6.3 Service Level Agreements

ID	Title	Requirement Details
NFR-LEG-006	Demonstration SLA	1. The platform is a demonstration prototype; no commercial service level is promised. 2. A production deployment shall define availability, response time and support levels with each lender.

5.7 Operational Requirements

5.7.1 Monitoring and Logging

ID	Title	Requirement Details
NFR-OPS-001	Health Checks	1. NIDA shall expose /api/v1/nida/health/ and Central shall expose /health/. 2. Server logs shall be written for each service.

5.7.2 Backup and Recovery Operations

ID	Title	Requirement Details
NFR-OPS-002	Backups	1. Database backups shall be taken before demonstrations and major changes. 2. Restore instructions shall be kept with the project.

5.7.3 System Administration

ID	Title	Requirement Details
NFR-OPS-003	Start and Stop	1. Scripts shall start and stop the services (setup, start, start-all, stop-all, reset).
NFR-OPS-004	Whitelists	1. The administrator shall add each caller's IP to the whitelist and restart the service. 2. Firewall rules shall open only the required ports.
NFR-OPS-005	Users	1. Administrator accounts shall be created with the framework commands and kept private.

5.7.4 Documentation Requirements

ID	Title	Requirement Details
NFR-OPS-006	Technical Documentation	1. The architecture, API and data contracts shall be documented and kept current. 2. The API shall be documented through Swagger.
NFR-OPS-007	User Documentation	1. Guides for customers, loan officers and Central operators shall be provided.

5.8 Cultural and Political Requirements

5.8.1 Cultural Considerations

ID	Title	Requirement Details
NFR-CUL-001	Inclusion	1. The design shall serve small-scale farmers, youth and women who may have no formal credit history. 2. Names shall support three name fields common in Tanzania.
NFR-CUL-002	Respect	1. Messages shall be polite and avoid terms that could stigmatize borrowers.

5.8.2 Political and Governance Requirements

ID	Title	Requirement Details
NFR-POL-001	Government Alignment	1. The system shall follow eGA standards and the national digital identity (NIDA) approach.
NFR-POL-002	Transparency	1. The scoring rules shall be public and verifiable in the smart contract. 2. The score shall be explained with its dimensions.
NFR-POL-003	Neutrality	1. All lenders shall receive the same result for the same borrower.

6. Other Requirements

6.1 Data Migration Requirements

ID	Title	Requirement Details
DM-001	NIDA Registry	1. The NIDA registry shall be loaded from the PostgreSQL dump (pg_restore) or from the fixture of 200 citizens. 2. The restored database shall contain the registry_citizen and registry_region tables.
DM-002	Lender Demo Data	1. Lender databases shall be filled with demonstration customers, accounts, loans and repayments using seed commands.
DM-003	Read Access	1. A read-only view (api_citizen) over active citizens may be given to a reader role for other systems.

6.2 Internationalization Requirements

ID	Title	Requirement Details
IR-001	Encoding	1. The system shall store and display UTF-8 text. 2. Names shall support three name fields.
IR-002	Currency and Time	1. Amounts shall be in TZS. 2. The time zone shall be Africa/Dar_es_Salaam.
IR-003	Language	1. The interfaces shall be in English; translation to Swahili is a future extension.

6.3 Training Requirements

ID	Title	Requirement Details
TR-001	Customers	1. Customers shall receive a short guide on registration, PIN and

ID	Title	Requirement Details
		OTP, consent and viewing their score.
TR-002	Loan Officers	1. Officers shall be trained to review applications, request the Central score and read the AI and blockchain results.
TR-003	Central Operators	1. Operators shall be trained to register lenders, run an assessment and broadcast the result.
TR-004	Technical Staff	1. Integrators shall be trained on the lender template, the whitelists and the deployment scripts.

6.4 Performance Requirements

ID	Title	Requirement Details
PR-001	Response Time	1. Ordinary pages shall respond within 3 seconds. 2. Scoring operations shall complete within 30 seconds.
PR-002	Availability	1. Services shall be available during demonstrations. 2. A failed lender shall not stop an assessment.
PR-003	Scalability	1. New lenders shall be added without changing Central code.
PR-004	Throughput	1. Each lender shall serve at least 50 concurrent users. 2. Central shall pull from all lenders concurrently.
PR-005	Resource Utilization	1. A backend shall run on a computer with 4 GB of memory.

6.5 Safety Requirements

ID	Title	Requirement Details
SR-001	Data Safety	1. User data shall be protected from unauthorized access. 2. Access control shall ensure only authorized users modify data.
SR-002	Backup and Recovery	1. Databases shall be backed up regularly. 2. Backups shall be tested by a restore.
SR-003	Financial Safety	1. Balances shall change only through recorded transactions. 2. Transfers and loan disbursements shall be atomic.
SR-004	User Privacy	1. The system shall comply with the Tanzania Data Protection Act, 2022. 2. Customers shall control data sharing through consent.

6.6 Security Requirements

ID	Title	Requirement Details
SEC-001	Authentication	1. Customers shall use account number, PIN and OTP. 2. Administrators shall use authenticated sessions with tokens. 3. PINs shall be stored as hashes.
SEC-002	Authorization	1. Role checks shall limit customers and administrators. 2. Systems shall be authorized by trusted IP address.
SEC-003	Data Protection	1. Personal data shall remain in PostgreSQL. 2. Only derived integers shall reach the blockchain. 3. Secrets shall not be stored in source control.

6.7 Software Quality Attributes

ID	Title	Requirement Details
QA-001	Usability	1. Interfaces shall be simple and consistent across the lender systems. 2. Validation messages shall be clear.
QA-002	Reliability	1. Business rules shall give the same result for the same input. 2. The smart contract shall be deterministic. 3. Audit trails shall record critical operations.
QA-003	Maintainability	1. Code shall be modular by Django app. 2. Automated tests shall cover the main flows.
QA-004	Scalability	1. Databases shall use PostgreSQL. 2. Systems shall run as separate processes.
QA-005	Security	1. All security requirements shall apply consistently across the systems.
QA-006	Transparency	1. Scoring rules and weights shall be public in the smart contract.

6.8 Business Rules

ID	Title	Requirement Details
BR-001	User Roles	1. Roles are customer and admin; each has specific permissions. 2. Administrators decide loans.
BR-002	Identity	1. A customer must be verified by NIDA before registration. 2. One NIDA number can be registered only once per lender.
BR-003	Consent	1. Data is shared with Central only with an active consent for the chosen categories.
BR-004	Data Ownership	1. Each lender owns its private data; Central holds the merged copy for audit and scoring.
BR-005	Score Use	1. The score is shared by all lenders for the same borrower. 2. Each lender applies its own lending rules.
BR-006	Score Rules	1. Score range 300–850; weights D1 30, D2 20, D3 15, D4 20, D5 15. 2. Minimum evidence: 6 months of history, 8 behaviour events, 1 repayment record and 1 verified source.
BR-007	Loan Terms	1. Default monthly interest rate is 18%; processing fee TZS 50,000.

7. Appendix

1. Glossary

Term	Definition
AI Engine	The component that analyses normalized credit features and returns the default probability, reputation and risk level.
Account Number	The number used with the PIN to log in as a customer; created at registration.

Term	Definition
API (Application Programming Interface)	A set of rules that lets software systems exchange data, such as the verify and receive endpoints of DAIRE.
Assessment	One scoring run for one borrower, identified by a unique assessment reference.
Assessment Reference	The unique key of one assessment (for example ASM-2026-0001).
Audit Log	A record of important actions with user, time, module and description.
Basis Points	Integers used instead of decimals; 10,000 basis points equal 100%.
Blockchain	A tamper-proof ledger on which the credit score is stored.
Borrower	A person who has or applies for a loan with one or more lenders.
Borrower Reference	The single cross-system key used by Central to match data of the same borrower.

Term	Definition
Broadcast	Sending the same assessment result from Central to every lender linked to the borrower.
Central System	The Web2 hub that receives lender data, merges it and coordinates AI and blockchain scoring.
Conflict	A difference between the values sent by two lenders for the same borrower.
Consent	The customer's permission to share selected data categories, with a start and expiry date.
Credit Score	A number from 300 to 850 calculated by the smart contract from five dimensions.
Cryptographic Erasure	Making on-chain data unlinkable by deleting the mapping to the person.
Customer ID	A unique identifier of a customer inside a lender system.
D1–D5 Dimensions	Payment reliability, financial stability, credit and debt management, behavioural consistency and verified trust evidence, each scored 0–100.
Data Exchange	A recorded operation between two systems, with direction, status and payload.
Data Routing Policy	The rule that lists which fields may be sent to lenders, AI and blockchain.
DAIRE	Decentralized AI Reputation Engine – the system described in this document.
Default Probability	The chance, from the AI model, that a loan becomes a default.
Django	A Python web framework used for the backends.

Term	Definition
DRF (Django REST Framework)	A toolkit used to build the REST APIs.
FSP (Financial Service Provider)	A bank, mobile-money or microfinance institution that lends money.

Term	Definition
Hardhat	A development environment that runs a local Ethereum network for the smart contract.
JWT (JSON Web Token)	A signed token used to prove the identity of a logged-in user.
Lender	A registered institution with its own lender system (CRDB-01, MoFinance-004, NMB-01, NMB-02).
Lender ID	The stable identifier of a lender in Central.
Lender Template	The common lender implementation that is duplicated for each new lender.
Loan Application	A customer request for a loan that waits for staff review.
Masked NIDA	A NIDA number shown with the middle digits hidden.
Middleware	The Express service that connects Central to the smart contract.
NIDA	National Identification Authority; issuer of the 20-digit national ID number.
NIDA Number	A 20-digit identity number: date of birth (YYYYMMDD) plus 12 digits.
OTP (One-Time Password)	A temporary code used at login and password reset; valid for 10 minutes.

Term	Definition
PIN	A 4-digit secret used with the account number to log in.
PostgreSQL	The relational database used by every system.
Random Forest	The machine-learning algorithm used for the AI reputation model.
Reputation	The AI label of the borrower: EXCELLENT, GOOD, MODERATE or HIGH_RISK.
Repayment Schedule	The list of installments of a loan with due dates and amounts.
REST	An API style based on HTTP requests and JSON responses.
Risk Band	The score class: NONE, POOR, FAIR, GOOD, VERY_GOOD or EXCEPTIONAL.
Routing	Deciding which data goes to which destination.
Smart Contract	A program on the blockchain (DaireCreditScore) that calculates and stores the score.
Solidity	The language in which the smart contract is written.
Sync Log	A record of each push from a lender to Central with its status.
TRUSTED_CALLBACK_IPS	The list of IP addresses allowed to call a lender's Central callback.
TRUSTED_CLIENT_IPS	The list of IP addresses allowed to call the NIDA system.

Term	Definition
TZS	Tanzanian shilling, the currency used in the system.
Unified Customer Data	The merged borrower record built in Central from all lenders.

Term	Definition
Web2 / Web3	Conventional web systems (Web2) and blockchain-based systems (Web3).
Weighted Sum	The sum of each dimension multiplied by its weight, maximum 10,000.
Verified Evidence	Data from sources that can be confirmed, used in the D5 dimension.